

# Tarea OpenMP

Jorge Grez

## 1 Función Zeta de Euler-Riemann (Problema 3)

### 1.1 Paralelización del bucle

Para paralelizar el ciclo principal (ver código), fue conveniente utilizar la directiva `#pragma omp parallel for`. Al revisar bien la estructura del código, nos podemos dar cuenta de que no existen dependencias de datos entre las distintas iteraciones del bucle externo. Básicamente, el cálculo que se hizo para obtener  $X[k]$  es totalmente independiente y no necesita saber qué pasó en iteraciones anteriores o posteriores, lo que lo hace ser un problema embarazosamente paralelizable. Vemos también que el puntero al arreglo  $X$ , nuestra variable real  $s$  y el límite superior  $N$  son variables compartidas. Todos los hilos pueden leer  $N$  y  $s$ , y todos escriben en el arreglo  $X$ . A pesar de que todos escriben en  $X$ , como cada hilo altera un índice  $k$  que es estrictamente exclusivo y distinto, evitamos de que se produzcan condiciones de carrera (race conditions) en la memoria. La variable iteradora  $k$  del bucle `for` se vuelve automáticamente privada para cada hilo, garantizando que cada proceso controle su propio avance de forma segura sin que se interfieran con los demás.

### 1.2 Balanceo de carga de los hilos

La complejidad asintótica de una sola llamada a la función es  $\mathcal{O}(k^2)$ . Esto significa que el tiempo de ejecución crece de forma cuadrática en relación con el valor de  $k$ . Si hacemos que OpenMP divida esto de forma estática y por partes iguales (por ejemplo si se tienen 4 hilos y  $N = 1000$ , darle un bloque continuo de 250 iteraciones a cada uno), sería un problema muy desbalanceado. El primer hilo procesaría los  $k$  más pequeños (del 0 al 249) de manera casi instantánea, mientras que el último hilo (del 750 al 999) enfrentaría una carga matemática muy pesada. Como consecuencia el hilo 1 terminaría rápidamente y se quedaría ocioso esperando a que el hilo 4 termine lo que le queda de cómputo.

### 1.3 Modos de Scheduling y tiempos de ejecución

Se probó el tiempo de ejecución para 3 distintos modos de scheduling y se derivó el siguiente análisis para cada planificación:

- **Planificación Estática:** Su tiempo de ejecución fue el más alto (2.62 seg promedio), esto probablemente sucedió porque los hilos se desincronizan rápidamente y dejan gran parte de la CPU inactiva esperando al hilo con los índices más altos. Por lo que se descartó totalmente.
- **Planificación Dinámica:** Su tiempo de ejecución fue de 1.07 seg promedio. Con este modo, el gestor asigna un nuevo valor de  $k$  a un hilo en cuanto este queda libre. Los hilos que procesen los valores de  $k$  cercanos a cero completarán sus tareas más rápido y volverán a la cola. Gracias a ello, los hilos rápidos ayudan a los que se atascan con los  $k$  grandes. Esto mantiene la ocupación de la CPU uniforme al 100% durante casi toda la simulación, minimizando drásticamente el tiempo de ejecución total.
- **Planificación guiada:** También dió un runtime excelente en este caso (1.06 seg promedio), siendo un poco mejor que en el modo dinámico y reduciendo un poco la sobrecarga que produce este modo por probablemente andar asignando tareas todo el tiempo.

## 2 Operaciones válidas para reducción paralela (problema 5)

Debemos recordar que para utilizar reducción paralela sobre una operación se deben cumplir las propiedades de asociatividad y conmutatividad. Teniendo eso en cuenta se procedió a verificar si las siguientes operaciones cumplen eso:

- **Máximo común divisor (GCD) de números enteros:** La operación es  $\gcd(a_0, a_1, \dots)$  la cual es asociativa ya que  $\gcd(a, \gcd(b, c)) = \gcd(\gcd(a, b), c)$  y también conmutativa ya que cumple  $\gcd(a, b) = \gcd(b, a)$  por lo que es una operación que puede beneficiarse de reducción paralela.
- **Producto de matrices generales de  $2 \times 2$ :** El producto entre matrices cuadradas es asociativo cumpliendo  $A(BC) = (AB)C$  pero en su caso general no cumple la conmutatividad ( $AB \neq BA$ ) por lo que no se puede implementar para una reducción paralela. Como los hilos terminan en distintos momentos, OpenMP podría multiplicar la matriz resultante del "Hilo 3" antes que la del "Hilo 1", alterando por completo el producto original de la secuencia.
- **Producto de números complejos:** El producto de números complejos cumple tanto asociatividad como conmutatividad por lo que sí se puede utilizar para reducción paralela.
- **Producto de cuaterniones:** Esta operación es asociativa pero no conmutativa ( $q_1 \cdot q_2 \neq q_2 \cdot q_1$ ) por lo que no se puede implementar para reducción paralela.

- **Evaluación incremental de promedios por pares:** Primero verificamos si la operación  $o : R \times R \rightarrow R$ ,  $(a, b) \mapsto \frac{a+b}{2}$  es asociativa:

$$o(a, o(b, c)) = o\left(a, \frac{b+c}{2}\right) = \frac{a + \frac{b+c}{2}}{2} = \frac{2a + b + c}{4} \quad (1)$$

$$o(o(a, b), c) = o\left(\frac{a+b}{2}, c\right) = \frac{\frac{a+b}{2} + c}{2} = \frac{a + b + 2c}{4} \quad (2)$$

Como  $\frac{2a+b+c}{4} \neq \frac{a+b+2c}{4}$ , la operación falla la propiedad. La falta de asociatividad destruye por completo el concepto de reducción, ya que dividir el trabajo en subgrupos entregará resultados distintos dependiendo de cómo se particionen los datos.